

KeenPay

Agentic commerce with bounded AI

KeenPay — architecture

System architecture

01 September 2026

<https://github.com/ambitiouss22/KeenPay>

KeenPay — architecture

FastAPI + LangGraph + PostgreSQL + Redis + Next.js + Razorpay.

Canonical architecture reference. Merges the production workflow spec, agentic-commerce V4 design (protocol gateway), and implementation build map.

Different protocols can enter KeenPay, but none can bypass the KeenPay Control Plane.

Engineering philosophy

AI models are non-deterministic. They are good at language, negotiation, and routing. They are not trusted to move money. KeenPay keeps a strict air-gap: agents and external protocols may propose commerce actions, but only the deterministic Control Plane may authorize and execute payment.

V4 adds a Protocol Gateway so UCP, ACP, AP2, A2A, MCP, x402, A2UI, and India rails (NPCI UAP / UPI) can enter through adapters — not through separate payment stacks. Every adapter normalizes to one KeenPay Intent. One policy engine. One audit trail.

Grow / Sell / Protect

Phase	Layer	Responsibility
Grow	AI / LangGraph	Intent parsing, catalog search, upsell & negotiation
Sell	Cart assembly	Cart assembler, price calculator (integer paise), checkout intent emitter
Protect	Control Plane	Policy + risk checks, authorization gate, payment execution

Trust boundaries

Level	What
Untrusted	Protocol payloads, user input, LLM output, inbound webhooks (validated on receipt)
Semi-trusted	LangGraph orchestration + protocol adapters (propose actions, never pay)
Trusted	Control Plane, integer math, scoped authorization, Razorpay client (gated side effects)

Planes

Plane	Role
Presentation	Next.js — chat panel + trace panel, WebSocket client
API gateway	FastAPI — REST, WS, protocol ingress, Razorpay webhook
Orchestration	LangGraph — <code>KeenPayStateGraph</code> , checkpoints in Postgres
Policy	Sync Python — <code>PolicyEngine</code> , no LLM in approve path
Data	Postgres source of truth; Redis cache, holds, rate limits, trace pub/sub

```
graph TD
    subgraph Client ["Next.js + A2UI surfaces"]
        Chat["Chat"]
        Trace["Trace"]
    end
    subgraph Gateway ["KeenPay API + Protocol Gateway"]
        REST["/api/v1"]
        WS["/ws/v1/session"]
        WH["/webhooks/razorpay"]
        Proto["Protocol adapters"]
    end
    subgraph Orchestration ["LangGraph"]
        Graph["KeenPayStateGraph"]
    end
    subgraph Policy ["Control Plane"]
        Rules["RuleEvaluator"]
        Anomaly["AnomalyScorer"]
    end
    subgraph Data ["Data"]
        PG["PostgreSQL"]
        RD["Redis"]
    end

    Chat <--> WS
    Trace <--> WS
    Proto --> Graph
    WS --> Graph
    REST --> Graph
    Graph --> Rules
    Rules --> PG
    Graph --> PG
    Graph --> RD
    Graph --> Razorpay
    WH --> PG
    Graph -->|trace publish| RD
    RD -->|subscribe| WS
```

Mermaid diagram source

Protocol Gateway (V4)

The Protocol Gateway is the single front door for agentic-commerce protocols. It authenticates the caller, validates schema, maps protocol-specific messages to a Normalized KeenPay Intent, and forwards that intent to the Control Plane. No adapter calls Razorpay, UPI, or x402 directly.

```
Protocol ingress (UCP | ACP | AP2 | A2A | MCP | x402 | A2UI | NPCI UAP / UPI)
-> Protocol Gateway (AuthN, agent identity, schema validation, replay / idempotency keys)
-> Protocol Adapters (thin translators -> Normalized KeenPay Intent)
-> Control Plane (Policy Engine -> Risk Scorer -> Authorization -> Inventory lock)
-> Payment rail router (Razorpay core; UPI / x402 stubs)
-> Settlement + audit (webhooks, reconciliation, Transaction Passport, audit_logs)
```

Protocol catalog

Protocol	Role	Status	KeenPay mapping
MCP	Controlled agent tools & context	Core	Tool allowlist on agents; MCP server maps to approved tools only

Protocol	Role	Status	KeenPay mapping
ACP	Agentic checkout / commerce	Core	Same as SELL path: cart -> guardrail -> pay
A2UI	Agent-driven UI components	Adapter-ready	Split UI: chat + trace; A2UI renders into hosted surfaces
UCP	Commerce interoperability	Adapter-ready	Adapter normalizes catalog/cart to KeenPay Intent
AP2	Payment mandates & verifiable auth	Adapter-ready	Maps to authorizations (scoped, expiring, single-use)
A2A	Agent-to-agent messaging	Experimental	Gateway accepts A2A envelope; no production adapter yet
x402	Machine / pay-per-use HTTP payments	Experimental	Rail router stub; does not bypass Control Plane
NPCI UAP	India unified agentic payments	Future	Architecture slot in rail router; no live integration
UPI	India instant payments	Future	Settlement rail behind same authorization object

What we do not do: separate payment systems per protocol; LLM-direct Razorpay / UPI / x402 calls; protocol-specific policy engines; claiming live UPI/x402 until adapter + rail tests pass.

Intended modules

Path	Responsibility
<code>frontend/</code>	Split UI, trace renderer
<code>api/main.py</code>	App entry, CORS, auth
<code>api/graph/keen_checkout.py</code>	Graph definition
<code>api/graph/state.py</code>	<code>KeenPayState</code>
<code>api/graph/edges.py</code>	Conditional routing
<code>api/policy/engine.py</code>	Guardrails
<code>api/policy/anomaly.py</code>	Injection + score
<code>api/services/catalog.py</code>	Product search
<code>api/services/razorpay.py</code>	Links + webhook verify
<code>api/services/audit.py</code>	<code>audit_logs</code> writes
<code>api/services/trace.py</code>	Redis pub/sub
<code>workers/webhook_processor.py</code>	Async idempotent webhook handling

Workflows: Grow, Sell, Protect

Lifecycle stays linear. Protocol origin does not shorten the path.

Protocol -> Gateway -> Intent -> GROW/SELL -> PROTECT -> [APPROVED] -> Pay -> Settle

Phase 1 — Grow (discovery & revenue optimization)

- 1 Intent — Buyer or external agent requests a product, bundle, or discount.
- 2 Catalog — Postgres full-text search — not LLM hallucination.
- 3 Upsell — Agent proposes bundle or discount percentage (not final price).
- 4 Intent emit — Adapter or LangGraph emits Normalized KeenPay Intent.

Phase 2 — Protect (guardrail interception)

- 1 Inventory lock — Redis + Postgres hold; fail closed if stock unavailable.
- 2 Policy — Max discount, margin floor, qty caps — Python only.
- 3 Price re-computation — Final total calculated in integer paise by backend, not the LLM.
- 4 Risk — Injection patterns, velocity, anomaly score.
- 5 Authorization — Scoped permission bound to cart hash + amount; single-use.

Phase 3 — Sell (Razorpay execution & settlement)

- 1 User confirm — Explicit confirm via LangGraph interrupt; not inferred from chat.
- 2 Idempotent order — Idempotency key per offer version; cart hash bound.
- 3 Payment link — Razorpay `POST /v1/payment_links` (test or live).
- 4 Webhook — HMAC verify, dedupe `event_id`, amount match, mark paid.
- 5 Passport — `audit_logs` + order snapshot for replay.

Request path: chat message

```
1. WS { type: "chat.message", text, session_id }
2. Validate JWT
3. Load/create checkpoint (langgraph_checkpoints / negotiation_sessions)
4. graph.ainvoke(..., config={ thread_id: session_id })
5. Each node -> TraceEvent -> Redis PUBLISH trace:{session_id}
6. WS forwards trace to client
7. chat.response with assistant message + structured offer if any
8. Persist negotiation_sessions + audit_logs on financial steps
```

Guardrail path (critical)

```

negotiate_offer -> proposed_offer (not trusted)
guardrail_check -> PolicyEngine.evaluate(...)
  -> guardrail_decision, guardrail_decision_id, approved_offer | rejection_reasons
  APPROVED -> compute_totals -> await_user_confirmation
  REJECTED -> explain_rejection -> negotiate if rounds < 5
  ESCALATED -> human_escalation -> escalation_tickets

```

Policy evaluation is synchronous. No LLM calls inside `guardrail_check`.

Payment path

```

await_user_confirmation (interrupt)
  user sends chat.confirm_payment
  create_payment_link:
    assert_payment_gates()
    reserve stock (Redis + PG transaction, inventory_holds)
    POST Razorpay /v1/payment_links (idempotency key)
    INSERT orders (pending)
    audit_logs PAYMENT_LINK_CREATED

```

Payment state machine

Same state machine for all rails. `UNKNOWN` is a real state — never blind retry.

State	Next	Notes
<code>created</code>	<code>payment_pending</code>	Order + authorization issued
<code>payment_pending</code>	<code>captured</code>	Webhook or poll confirms pay
<code>captured</code>	<code>completed</code>	Reconciliation OK
<code>payment_pending</code>	<code>unknown</code>	Timeout / no webhook
<code>unknown</code>	<code>reconciliation</code>	Ask provider truth
<code>unknown</code>	<code>captured / failed</code>	After reconcile
<code>authorization</code>	<code>approved -> consumed</code>	Single-use scope
<code>authorization</code>	<code>expired / revoked</code>	No pay allowed

Webhook + reconciliation (all rails):

- Verify signature (HMAC for Razorpay)
- Dedupe by `provider_event_id`
- Amount must match `authorized amount_minor`
- Mismatch -> `payment_disputed`, HITL P0, no auto-complete
- Outbox worker publishes events; reconciliation worker resolves `UNKNOWN`

Webhook path

```

POST /webhooks/razorpay
verify X-Razorpay-Signature
INSERT webhook_events (event_id UNIQUE) – duplicate -> 200 no-op
match order by payment_link_id
if amount != order.final_amount_paise -> payment_disputed
else orders.status = paid, audit_logs PAYMENT_CAPTURED
WS order.status_updated

```

State shape (KeenPayState)

Defined in [api/graph/state.py](#):

```

class KeenPayState(TypedDict):
    messages: Annotated[list, add_messages]
    session_id: str
    user_id: Optional[str]
    merchant_id: str
    parsed_intent: Optional[dict]
    search_results: list[dict]
    selected_line_items: list[LineItem]
    proposed_offer: Optional[ProposedOffer]
    approved_offer: Optional[ProposedOffer]
    negotiation_round: int # max 4 (0-indexed) -> 5 rounds
    guardrail_decision: Optional[Literal["APPROVED", "REJECTED", "ESCALATED"]]
    guardrail_decision_id: Optional[str]
    guardrail_detail: Optional[GuardrailDecision]
    rejection_reasons: list[str]
    user_confirmed_payment: bool
    user_confirmed_at: Optional[str]
    final_amount_paise: Optional[int]
    inventory_reserved: bool
    razorpay_payment_link_id: Optional[str]
    razorpay_payment_link_url: Optional[str]
    order_id: Optional[str]
    anomaly_flags: list[str]
    security_block: bool
    error: Optional[dict]

```

Python

[ProposedOffer](#) / [GuardrailDecision](#) models are in the same module (see [api/graph/state.py](#)).

Graph topology

```
stateDiagram-v2
    [*] --> parse_intent
    parse_intent --> catalog_search: ok
    parse_intent --> clarify_intent: low confidence
    catalog_search --> negotiate_offer: found
    catalog_search --> suggest_alternatives: empty
    negotiate_offer --> guardrail_check
    guardrail_check --> compute_totals: APPROVED
    guardrail_check --> explain_rejection: REJECTED
    guardrail_check --> human_escalation: ESCALATED
    explain_rejection --> negotiate_offer: rounds left
    compute_totals --> await_user_confirmation
    await_user_confirmation --> create_payment_link: confirmed
    create_payment_link --> dispatch_success: ok
    create_payment_link --> payment_link_failed: error
```

Mermaid diagram source

Edge helpers (api/graph/edges.py)

```
def after_guardrail(state: KeenPayState) -> str:
    d = state.get("guardrail_decision")
    if d == "APPROVED":
        return "approved"
    if d == "ESCALATED":
        return "escalated"
    return "rejected"

def after_rejection(state: KeenPayState) -> str:
    if state.get("negotiation_round", 0) >= 5:
        return "max_rounds"
    return "retry_negotiate"
```

Python

Checkpointing

- Checkpointer: `AsyncPostgresSaver` on `langgraph_checkpoints`
- `thread_id = negotiation_sessions.id`
- Interrupt before `await_user_confirmation`
- Resume: `Command(resume={"user_confirmed_payment": True})`
- Guardrail decisions bind to `(session_id, offer_version)`

Security matrix

No protocol or LLM prompt overrides these. Enforced in Control Plane code.

Vector	Threat	KeenPay response
Cross-tenant leak	Protocol sends wrong tenant	RLS + server-set tenant context
Double spend	Retry after timeout	Idempotency + never-retry-unknown
Margin erosion	Injection / fake discount	<code>RULE_MIN_MARGIN</code> + price floor
Oversell	Two agents, one SKU	Redis lock + <code>FOR UPDATE</code> holds

Vector	Threat	KeenPay response
Untraceable pay	Dispute with no proof	Transaction Passport from <code>audit_logs</code>
Forged webhook	Fake paid event	HMAC + event dedupe + amount check
LLM math	Wrong total	Integer paise in <code>compute_totals</code> only
Protocol bypass	Adapter calls Razorpay	Adapters cannot hold payment credentials
Prompt injection	Bypass rules via chat	Deterministic regex + anomaly scorer; <code>security_block</code> halts money actions
Data isolation	Cross-merchant data leaks	Row-Level Security in PostgreSQL
Concurrency	Two agents sell last item	Redis distributed locks + <code>inventory_holds</code>

Authorization outcomes

Outcome	When
AUTO-APPROVE	Low risk: discount within policy, stock available, anomaly score < 0.5
CLAMP	Discount exceeds cap: force to <code>merchant.max_discount_limit</code>
REJECT	Margin violation or invalid SKU: halt, explain to user
STEP-UP / HUMAN	High risk or max negotiation rounds: escalation ticket to merchant

Protocol Gateway security

Every protocol adapter runs the same security checks before the Control Plane sees data:

Control	Gateway enforcement
Authentication	mTLS or signed JWT per protocol; no anonymous money ingress
Agent identity	<code>agents</code> table: type, scopes, <code>allowed_tools</code> , <code>trust_level</code> , expiry
Schema validation	Pydantic / JSON Schema per adapter; reject malformed payloads
Replay protection	nonce + timestamp window; <code>idempotency_keys</code> per scope
Scoped authorization	AP2 mandates map to authorizations; bound to <code>cart_hash</code> + amount
Idempotency	Same key returns same result; no double spend
Tool allowlisting	MCP tools must appear in <code>agents.allowed_tools</code> JSONB
Tenant isolation	<code>tenant_id</code> on every row; Postgres RLS; <code>SET LOCAL</code> per request

Transaction Passport & audit

No separate passport table. Built from `negotiation_sessions`, `orders`, `audit_logs`, and `webhook_events` for support replay and compliance.

Field	Source
<code>protocol_ref</code>	Which adapter / protocol originated the intent
<code>session_id</code>	Agent session or LangGraph thread
<code>authorization_id</code>	Scoped approval used for payment
<code>decision_id</code>	Guardrail evaluation reference
<code>offer_version</code>	Monotonic cart proposal version
<code>policy_version</code>	Ruleset that ran (e.g. 2026.08.1)
<code>grow_trace</code>	Intent, catalog hits, negotiation rationale
<code>protect_trace</code>	Per-rule pass/fail/clamp results
<code>sell_trace</code>	Rail, <code>idempotency_key</code> , webhook ids
<code>final_amount_paise</code>	Deterministic integer total

Live (Redis): `PUBLISH trace:{session_id}`

```
class TraceEvent(BaseModel):
    event_id: str
    session_id: str
    timestamp: str
    event_type: Literal[
        "graph.node.enter", "graph.node.exit",
        "guardrail.rule.eval", "guardrail.decision",
        "security.flag", "payment.link.created",
        "payment.webhook.received", "error",
    ]
    node_name: Optional[str]
    payload: dict
    duration_ms: Optional[int]
```

Python

Durable (Postgres): `audit_logs` — append only, trigger blocks UPDATE/DELETE.

Every money step logs `session_id`, `decision_id`, `offer_version` where applicable.

Redis keys

Key	TTL	Use
<code>session:{id}:messages</code>	24h	hot cache
<code>hold:{session_id}:{sku}</code>	15m	soft hold
<code>ratelimit:payment_link:{session_id}</code>	1h	max 3 links/hour
<code>ratelimit:chat:{user_id}</code>	1m	30 msg/min
<code>idempotency:rz:link:{session_id}:v{n}</code>	24h	cached link response

Deploy

Pilot (current)

```
Vercel / static host -> Next.js
Railway / Fly        -> FastAPI + LangGraph
Managed             -> Postgres 15, Redis 7
External             -> Razorpay, LLM provider
```

Env: `DATABASE_URL`, `REDIS_URL`, `RAZORPAY_*`, `OPENAI_API_KEY`, `MERCHANT_POLICY_JSON`.

AWS (production target)

Monolithic API first. Workers for webhooks and reconciliation. No per-protocol stacks.

Layer	AWS service	Purpose
Edge	ALB + WAF	TLS termination, rate limits
Compute	ECS Fargate / EKS	FastAPI + LangGraph + Protocol Gateway
Data	RDS PostgreSQL	RLS catalog, orders, <code>audit_logs</code>
Cache	ElastiCache Redis	Locks, sessions, trace pub/sub
Secrets	Secrets Manager	Razorpay keys — never in DB or LLM
Async	SQS + workers	<code>webhook_events</code> , outbox, reconciliation
Static	S3 + CloudFront	A2UI assets, exports
Logs	CloudWatch	Structured audit <code>correlation_id</code>

Human-in-the-loop

- `escalation_tickets` for ESCALATED guardrail outcomes
- Human override logged with `actor=human`; margin floor not overridden in v1
- Payment disputed (webhook mismatch) -> P0 queue

Security notes

- LLM -> `ProposedOffer` only; amounts need guardrail pass
- Webhook: HMAC on raw body; reject stale (>5 min skew)
- WS: JWT in query, validated on connect
- No email/phone in trace payloads

Failure summary

Failure	Response
LLM timeout	10s cap; no offer change
DB down	503
Guardrail exception	ESCALATED + audit

Failure	Response
Razorpay 5xx	3 retries exponential
Duplicate webhook	200 idempotent

Full matrix: [GUARDRAILS_AND_SAFETY.md](#).

AI boundary

LLM: [parse_intent](#), [negotiate_offer](#) copy. Python: everything that touches money.

Do not add [create_payment_link](#) as an LLM tool. Always call [assert_payment_gates\(\)](#) before Razorpay.

See [AI_JUDGMENT.md](#).

V4 summary (August 2026)

KeenPay V4 welcomes the agentic-commerce protocol ecosystem through one Protocol Gateway and one Control Plane. GROW and SELL are unchanged in spirit. PROTECT is unchanged in law: policy, risk, and authorization run in deterministic Python before any rail is called.

Implementation truth:

- Core today: MCP tool gating, ACP/SELL checkout, Razorpay, audit, RLS, HITL design
- Adapter-ready: UCP, AP2, A2UI gateway contracts documented
- Experimental: A2A ingress envelope, x402 rail stub
- Future: NPCI UAP / UPI settlement adapters